

RELATÓRIO FINAL DE ATIVIDADES DE INICIAÇÃO CIENTÍFICA

**Comunicação e Padronização SunSpec para
BMS de Baterias de Lítio via Modbus RTU**

vinculado ao projeto

**CONSOLIDAÇÃO DO NOVO ARRANJO DE PESQUISA EM
SUSTENTABILIDADE E QUALIDADE DE VIDA / NAPI SUDOESTE**

Lucas Santos de Lucca

Bolsista DIRPPG/Fundação Araucária

Engenharia de Computação

Data de Ingresso no Programa: 07/2025

Orientador: Prof. Dr. Gustavo Weber Denardin

Área do Conhecimento: 3.04.05.00-9 — Engenharias

CAMPUS PATO BRANCO, 2026

LUCAS SANTOS DE LUCCA   
GUSTAVO WEBER DENARDIN   

**COMUNICAÇÃO E PADRONIZAÇÃO SUNSPEC PARA
BMS DE BATERIAS DE LÍTIO VIA MODBUS RTU**

Relatório de Pesquisa do Programa de Iniciação Científica da Universidade Tecnológica Federal do Paraná.

PATO BRANCO, 2026

SUMÁRIO

RESUMO 3

INTRODUÇÃO 3

MATERIAL E MÉTODOS 4

PROJETO E DESENVOLVIMENTO DE HARDWARE 4

ARQUITETURA DE FIRMWARE E CAMADA DE ENLACE 7

PADRONIZAÇÃO DE DADOS E SERVIDOR SUNSPEC 7

RESULTADOS E DISCUSSÃO 7

CONCLUSÕES 9

AGRADECIMENTOS 10

REFERÊNCIAS 10

RESUMO

Este trabalho apresenta o desenvolvimento de uma arquitetura de hardware e firmware para o monitoramento e a coleta padronizada de dados de Sistemas de Gerenciamento de Baterias (BMS) de íons de lítio, especificamente da família Unipower UPLFP48. O principal objetivo consistiu em estabelecer a comunicação com o equipamento via protocolo Modbus RTU e padronizar as informações obtidas utilizando modelos SunSpec Modbus. A metodologia abrangeu o projeto e a fabricação de placas de circuito impresso para o acoplamento de transceptores RS-485, aliados ao desenvolvimento de um firmware de tempo real baseado no sistema operacional FreeRTOS. Para mitigar problemas de latência e ruído inerentes ao barramento de comunicação físico, implementou-se uma técnica de recepção assíncrona baseada em Acesso Direto à Memória (DMA), em conjunto com técnicas de *type punning* para a interpretação direta dos registradores em estruturas de dados. Os resultados demonstraram o sucesso na aquisição determinística e na conversão dos dados de um nó de bateria, validados por meio de uma Interface de Linha de Comando para diagnóstico em tempo real. Conclui-se que o sistema apresenta a robustez necessária para homologar o nó principal, estabelecendo a infraestrutura para a futura integração de múltiplos conjuntos em topologia *daisy chain*.

Palavras-chave: Baterias de Lítio; Modbus RTU; SunSpec; FreeRTOS; Sistemas Embarcados.

INTRODUÇÃO

A transição energética global e a busca por eficiência em sistemas de geração distribuída têm impulsionado a adoção de microrredes e sistemas de armazenamento de energia. Nesse cenário, bancos de baterias de íons de lítio desempenham um papel crucial. Contudo, para que operem com segurança, estabilidade e máxima vida útil, é indispensável o uso de um Sistema de Gerenciamento de Baterias. Um dos grandes desafios na integração desses sistemas de armazenamento é a falta de interoperabilidade na comunicação entre equipamentos de diferentes fabricantes. O protocolo MODBUS, operando sobre o padrão de camada física multiponto RS-485, consolidou-se como um padrão de fato na indústria desde 1979 para a comunicação cliente/servidor entre dispositivos de automação. Apesar de a camada de transporte e o protocolo serem padronizados, o mapeamento de dados é, na maioria das vezes, proprietário. Isso exige o desenvolvimento de drivers customizados para cada modelo de bateria, encarecendo a integração e dificultando a escalabilidade [1]. Para mitigar esse problema, a SunSpec Alliance, uma aliança comercial de desenvolvedores e fabricantes, busca estabelecer padrões abertos de informação para a indústria de energia distribuída. A adoção desses

modelos padroniza a troca de informações operacionais de sistemas fotovoltaicos e de armazenamento, reduzindo custos, promovendo a inovação e acelerando o crescimento do setor. A padronização SunSpec organiza o mapa de registradores Modbus iniciando por um identificador fixo e um Modelo Comum, focado na identificação do fabricante e do dispositivo, seguido por modelos de domínio específico. Para sistemas de armazenamento de energia, utiliza-se o Model 802 [2][3]. O presente trabalho de Iniciação Científica tem como foco o desenvolvimento de uma interface de comunicação e padronização para o BMS de baterias de lítio comerciais, família Unipower UPLFP48. Para garantir eficiência computacional e compatibilidade com o padrão industrial, a solução embarcada utiliza um microcontrolador STM32F103C8T6 [4], programado em C com as bibliotecas nativas HAL do ambiente CubeIDE. Como a arquitetura de rede em barramento RS-485 exige adequação elétrica rigorosa, faz-se necessário o projeto de um hardware dedicado integrando transceptores MAX3486 [5]. Diante do contexto apresentado, as atividades e objetivos a serem desenvolvidos por este projeto compreendem:

- Implementar a comunicação de microcontroladores com conjuntos de baterias comerciais utilizando o protocolo Modbus RTU;
- Padronizar as informações dos dados lidos da bateria em modelos SunSpec Modbus;
- Implementar um servidor SunSpec Modbus para a coleta padronizada dos dados da bateria;
- Desenvolver placas de circuito impresso para conectar os microcontroladores aos transceptores de comunicação da rede RS-485.

MATERIAL E MÉTODOS

O desenvolvimento deste trabalho foi conduzido em três frentes principais interconectadas: o projeto de hardware para a interface física de comunicação, a arquitetura de firmware de tempo real para o microcontrolador e a implementação da camada de padronização de dados. A seguir, detalham-se os materiais e as metodologias empregadas em cada uma dessas etapas.

PROJETO E DESENVOLVIMENTO DE HARDWARE

Para estabelecer a comunicação física com o Sistema de Gerenciamento de Baterias, adotou-se o padrão elétrico RS-485. O hardware central do nó de comunicação foi baseado no microcontrolador STM32F103C8T6 [4], arquitetura ARM Cortex-M3, integrado em uma placa de desenvolvimento comercialmente conhecida como Blue Pill. Para a conversão dos níveis lógicos da UART do microcontrolador para os níveis

diferenciais do barramento RS-485, utilizou-se o transceptor MAX3486 [5].

Para a conexão com o barramento do BMS, a saída diferencial do MAX3486 [5] foi roteada para um conector RJ45. Em conformidade com o padrão internacional de pinagem estabelecido pela *Modbus Organization* para cabos de par trançado, o sinal D0 (linha A, não-inversora) foi alocado no pino 5, enquanto o sinal D1 (linha B, inversora) foi alocado no pino 4 [1].

O projeto da Placa de Circuito Impresso Foi desenvolvido utilizando software de automação de design eletrônico, nesse caso o Kicad [6]. A modelagem abrangeu o desenho do esquemático elétrico, definindo as conexões de alimentação, os resistores de terminação e polarização da rede RS-485, e o roteamento das trilhas presente na Figura 1.

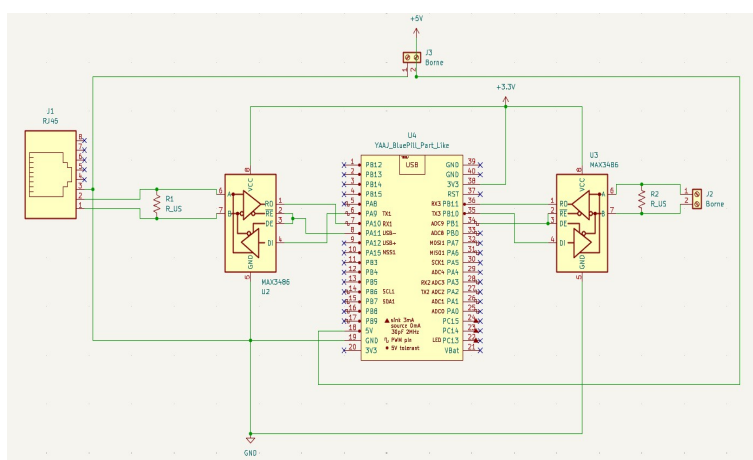


Figura 1. Esquemático elétrico da interface de comunicação RS-485.

Fonte: Autoria própria

Após o roteamento, gerou-se o modelo PCI da placa para validação elétrica e disposição dos componentes presente na Figura 2.

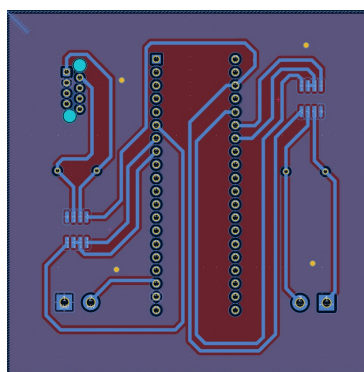


Figura 2. Modelo PCI projetado.

Fonte: Autoria própria

Após a validação, gerou-se o modelo 3D da placa para visualização presente na Figura 3.

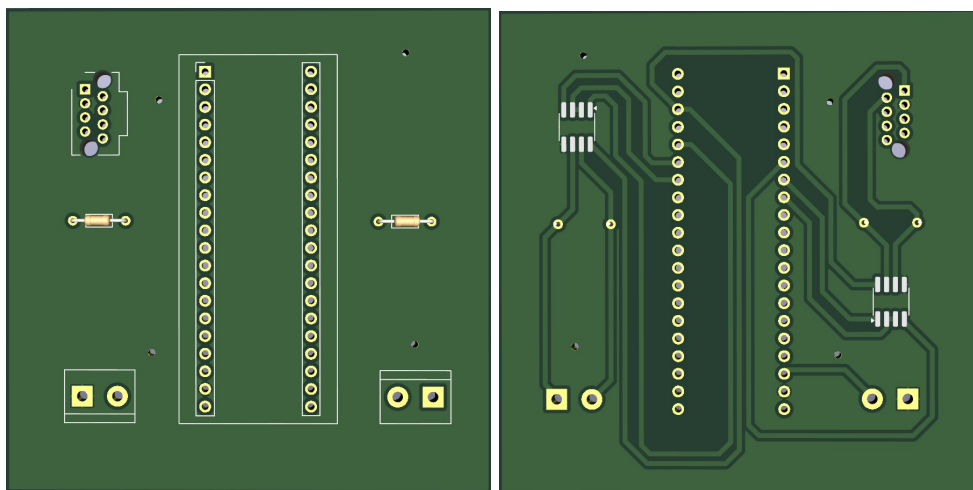


Figura 3. Modelo 3D projetado.

Fonte: Autoria própria

A etapa de prototipagem consistiu na fabricação da PCB e na soldagem manual dos componentes, culminando na montagem experimental do hardware final utilizado para os testes de bancada presente na Figura 4.

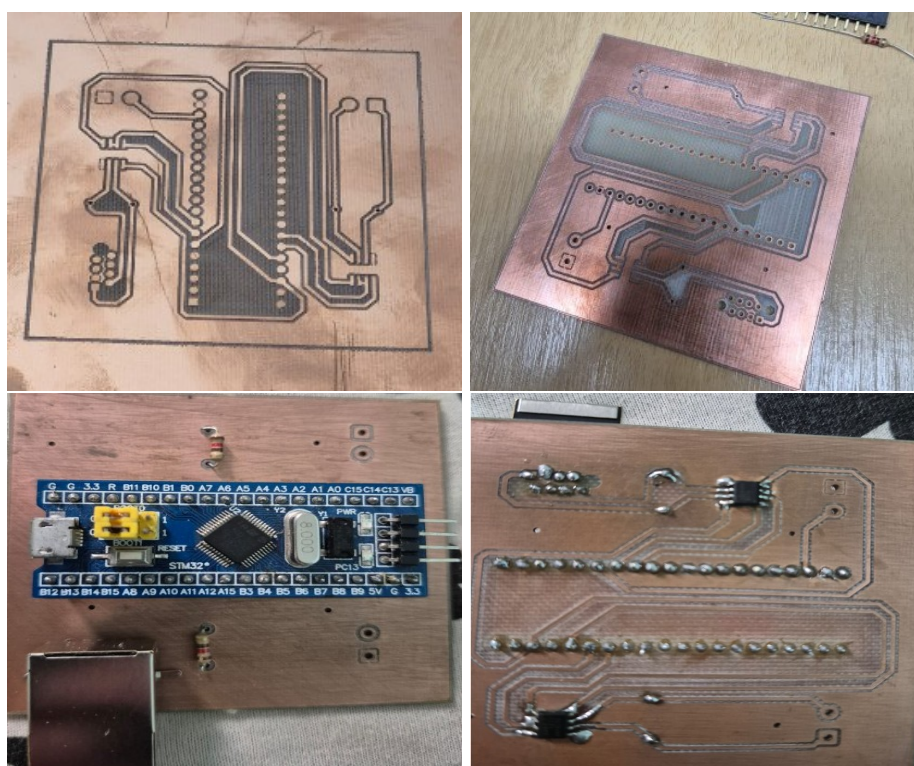


Figura 4. Protótipo pronto.

Fonte: Autoria própria

ARQUITETURA DE FIRMWARE E CAMADA DE ENLACE

O firmware foi desenvolvido em linguagem C utilizando a IDE STM32CubeIDE e as bibliotecas de abstração de hardware. Para lidar com os requisitos de concorrência e evitar o bloqueio da CPU durante a espera por dados no barramento serial, o sistema operacional de tempo real FreeRTOS [7] foi embarcado no microcontrolador.

A comunicação Modbus RTU, devido à sua natureza determinística e suscetibilidade a ruídos elétricos no barramento físico, foi implementada utilizando Acesso Direto à Memória. Ao invés de processar interrupções a cada byte recebido na porta UART, o controlador DMA foi configurado para transferir o payload Modbus bruto [8] diretamente para a memória RAM. Durante esse período, a tarefa de leitura da bateria no FreeRTOS entra em estado de suspensão, consumindo o mínimo de ciclos de CPU. Após o tempo estipulado para a recepção do pacote, a tarefa é retomada, valida o cabeçalho Modbus e realiza o cálculo de redundância cíclica para atestar a integridade da mensagem. Pacotes corrompidos são descartados, garantindo a estabilidade da máquina de estados do sistema.

PADRONIZAÇÃO DE DADOS E SERVIDOR SUNSPEC

Com a comunicação Modbus RTU estabelecida, o microcontrolador realiza a extração dos dados brutos em formato hexadecimal, com extremidade big-endian. A etapa metodológica seguinte consistiu em decodificar, converter e mapear essas informações para os Modelos de Informação SunSpec.

Conforme a especificação SunSpec, implementou-se o Model 1 para a identificação do nó de comunicação, contendo fabricante, modelo e versão. E o Model 802, específico para sistemas de armazenamento de energia [2][3].

Para mitigar o overhead de processamento na conversão de dezenas de registradores, empregou-se a técnica de *type punning* em C. Através do uso de union e da diretiva de pré-compilação, o mapa de registradores SunSpec foi sobreposto exatamente ao mesmo endereço físico de memória do vetor de recepção Modbus. Isso eliminou a necessidade de rotinas custosas de cópia de memória e deslocamento de bits, permitindo que a estrutura de dados padronizada fosse populada quase instantaneamente em tempo constante, pronta para ser disponibilizada ao servidor ou coletor de dados da microrrede.

RESULTADOS E DISCUSSÃO

Os testes de validação da arquitetura proposta foram realizados em bancada, interligando a Placa de Circuito Impresso desenvolvida, contendo o microcontrolador

e o transceptor MAX3486, diretamente à porta de comunicação RS-485 do BMS da bateria Unipower UPLFP48.

Para atestar a integridade da camada física e o determinismo do firmware, utilizou-se a Interface de Linha de Comando embarcada. Através de um terminal serial (PuTTY), foi possível enviar comandos assíncronos ao microcontrolador e verificar a resposta do sistema em tempo real, sem causar bloqueios na tarefa principal de leitura do FreeRTOS. A captura de tela apresentada na Figura 5 demonstra a execução bem-sucedida da CLI.

```

COM3 - PuTTY
MODBUS RTU - Baterias Litio Unipower UPLFP48

UTFPR CLI - Digite 'help'
> help
Comandos: status, batt, modbus, regs, all, clear

> status
Comm Status: OK (Err:99)

> batt
Batt Voltage: 49.29 V

> modbus
--- DUMP MODBUS (Primeiros 10 regs) ---
Reg[00]: 0x1340
Reg[01]: 0x0000
Reg[02]: 0x0CD3
Reg[03]: 0x0CD4
Reg[04]: 0x0CD6
Reg[05]: 0x0CD6
Reg[06]: 0x0CD3
Reg[07]: 0x0CD5
Reg[08]: 0x0CD8
Reg[09]: 0x0CD7

> regs
--- INTERPRETACAO DOS REGISTRADORES ---
Voltage Total: 49.28 V
Corrente:      0.0 A
Temp. Interna: 22 C
SoC:          99 %

--- TENSÃO DAS CELULAS ---
C01: 3.283 V | C02: 3.284 V | C03: 3.286 V | C04: 3.286 V
C05: 3.283 V | C06: 3.285 V | C07: 3.288 V | C08: 3.287 V
C09: 3.287 V | C10: 3.287 V | C11: 3.283 V | C12: 3.288 V
C13: 3.287 V | C14: 3.288 V | C15: 3.287 V | C16: 0.000 V

> all
--- RELATORIO COMPLETO ---
SoH:100% | SoC:99%

> clear

```

Figura 5. Interface de Linha de Comando demonstrando a leitura dos dados do BMS.

Fonte: Autoria própria

Conforme ilustrado na Figura 5, o comando status retornou o código de erro 99, definido na máquina de estados da aplicação como indicativo de sucesso absoluto, ausência de ruídos ou perdas de sincronismo no barramento. Adicionalmente, o comando modbus permitiu a visualização do dump hexadecimal bruto extraído diretamente da camada de transporte, atestando que o nó realizou a requisição e a recepção do pacote completo com sucesso.

A segunda etapa de validação concentrou-se na camada de aplicação, especificamente na técnica de *type punning* utilizada para mapear os registradores Modbus para as estruturas padronizadas. A Figura 6 apresenta o ambiente de depuração, exibindo a

alocação de memória durante a execução do firmware.

Expression	Type	Value
▼ batt	modbus_battery_t	{...}
> regs	uint16_t [28]	[28]
▼ data	battery_data_t	{...}
⌘ voltage_dV	uint16_t	4928
⌘ current_dA	int16_t	0
> cel_voltage	uint16_t [16]	[16]
⌘ bms_cooling_temp	int16_t	22
⌘ battery_internal	int16_t	22
⌘ temp_max	int16_t	22
⌘ remaining_cap	uint16_t	99
⌘ max_recharge	uint16_t	100
⌘ soh	uint16_t	100
⌘ soc	uint16_t	99
⌘ status	uint16_t	0
⌘ alarm	uint16_t	0
⌘ protection	uint16_t	0
▼ modbus_regs	uint16_t [256]	[256]
> [0..99]		[256]
> [100..199]		[256]
> [200..255]		[256]
> batt_rx_buf	uint8_t [80]	[80]
⌘ batt_err_code	uint8_t	99 'c'
⌘ batt.data.voltage_dV	uint16_t	4928
+ Add new expression		

Figura 6. Depurador de memória demonstrando o mapeamento direto via union.

Fonte: Autoria própria

Observa-se na Figura 6 que o vetor bruto de recepção de 80 bytes (batt_rx_buf) foi corretamente transferido via DMA. Através da union implementada, esses bytes preencheram instantaneamente os campos nominais da estrutura battery_data_t. A conversão foi validada com sucesso, como evidenciado pelos campos voltage_dV (4928, correspondente a 49,28 V), state of charge (99), state of health (100) e remaining_cap (99 Ah). Os valores condizem perfeitamente com o estado físico de uma bateria em flutuação térmica e com a conversão de escala requerida pelo padrão de modelagem.

Por fim, a estabilidade ininterrupta alcançada nesta etapa atinge o marco fundamental do projeto. A validação deste nó de comunicação primário, operando com total integridade de dados e tratamento de erros mitigado via software, homologa a arquitetura desenvolvida. Este cenário consolida a infraestrutura necessária para, em trabalhos futuros, expandir o barramento RS-485 para a topologia *daisy chain*, viabilizando a comunicação simultânea e a padronização SunSpec para os demais conjuntos de baterias da microrrede.

CONCLUSÕES

Este projeto de Iniciação Científica cumpriu integralmente os objetivos estabelecidos para a fase de prototipagem e validação da comunicação de baterias de lítio em

microrredes. O desenvolvimento e a montagem da Placa de Circuito Impresso com transceptores MAX3486 forneceram a robustez elétrica necessária para a operação no barramento físico RS-485. No âmbito do software, a implementação da arquitetura baseada no sistema operacional de tempo real FreeRTOS e no Acesso Direto à Memória garantiu uma comunicação Modbus RTU determinística, imune a bloqueios de CPU.

A decodificação dos dados brutos demonstrou que a técnica de alocação direta de memória é um método altamente eficiente para popular as estruturas padronizadas. Os testes práticos confirmaram a correta leitura e interpretação das grandezas físicas da bateria Unipower UPLFP48 (tensão, corrente, SoC, SoH), mapeando-as com exatidão para o escopo do Model 1 e Model 802 da especificação SunSpec Modbus. O nó de comunicação primário desenvolvido atende plenamente aos requisitos de interoperabilidade esperados para conversores estáticos e servidores em sistemas de geração distribuída.

Como desdobramento natural desta pesquisa, sugere-se, para trabalhos futuros, a expansão do sistema validado para uma rede com topologia em cascata, *daisy chain*. A infraestrutura de hardware e firmware homologada neste relatório servirá de base para a comunicação simultânea e a coleta de dados agregados de múltiplos conjuntos de baterias no mesmo barramento, viabilizando o monitoramento em larga escala de microrredes de energia.

AGRADECIMENTOS

Os autores agradecem a Fundação Araucária e NAPI-SUDOESTE, pelo apoio financeiro através de bolsa de iniciação científica, e a Universidade Tecnológica Federal do Paraná, pela estrutura laboratorial, disponibilização de material de consumo e equipamentos. Além da ajuda dos colegas acadêmicos Ezequiel Gonschorowski e Célio Antônio Degaraes na fabricação e desenvolvimento da PCB.

- O presente trabalho foi realizado com apoio da Fundação Araucária — Brasil — Edital DIRPPG-PB/UTFPR n.º 05/2025 NAPI-SUDOESTE.

REFERÊNCIAS

- [1] MODBUS ORGANIZATION. **MODBUS over Serial Line Specification and Implementation Guide**. V1.02. North Grafton, MA, USA, 2006.
- [2] SUNSPEC ALLIANCE. **SunSpec Information Model Specification**. Version 1.9. San Jose, CA, USA, 2015.
- [3] SUNSPEC ALLIANCE. **SunSpec Device Information Model Specification**. Version

1.1. San Jose, CA, USA, 2022.

[4] STMICROELECTRONICS. **STM32F103x8/B Datasheet - Mainstream Performance line, ARM Cortex-M3 MCU**. Rev 17. Geneva, Switzerland, 2015.

[5] ANALOG DEVICES. **MAX3483-MAX3491: 3.3V-Powered, True RS-485/RS-422 Transceivers**. Rev 1. Wilmington, MA, USA, 2019.

[6] KICAD DEVELOPERS TEAM. **KiCad EDA Software Suite**. Version 8.0. Disponível em: <https://www.kicad.org>. Acesso em: 04 mai. 2026.

[7] AMAZON WEB SERVICES. **FreeRTOS: Real-time operating system for micro-controllers**. Document Version 2024.10.00. Seattle, WA, USA, 2024.

[8] MODBUS ORGANIZATION. **MODBUS Application Protocol Specification**. V1.1b3. North Grafton, MA, USA, 2012.